



Modélisation SystemVerilog de l'algorithme de chiffrement ASCON-AEAD128

Conception de Systèmes Numériques

Professeur :

Guillaume REYMOND

Élève :

Lisa- Marie DURIEUX

Mai 2025

Table des matières

<u>Introduction</u>	<u>3</u>
Présentation de l'algorithme ASCON	3
Déroulé du projet de conception de l'algorithme ASCON	3
<u>Fonctionnement global de l'algorithme de chiffrement ASCON</u>	<u>4</u>
<u>Modélisation de l'algorithme ASCON</u>	<u>7</u>
Modélisation des fonctions élémentaires de la permutation	7
Modélisation de la permutation (sans XOR)	11
Modélisation de la permutation avec les XOR	13
Modélisation de la machine d'état finis (FSM) - Machine de Moore	16
Résultats d'implémentation	19
<u>Difficultés rencontrées</u>	<u>20</u>
Erreurs de Syntaxe	20
La permutation	20
La Machine d'états finis	20
Debug des modules	20
<u>Conclusion</u>	<u>21</u>

Introduction

Les objets connectés, issus de l'internet des objets (IoT), occupent depuis quelques années une place importante de notre quotidien. Qu'ils soient présents dans les logements avec la domotique, dans le domaine de la santé avec les outils de suivi médical connectés, ou encore dans notre quotidien avec les montres ou autres gadgets connectés, ils permettent de faciliter certaines tâches de tous les jours.

Cependant, certains de ces objets ont été la cible d'attaques en ligne, notamment pour voler des données sensibles stockées dans le *cloud* ou leur mémoire interne. En effet, ces objets ayant d'abord été conçus et optimisés pour avoir une faible consommation énergétique, afin d'être aussi autonome et nomade que possible (dans la plupart des cas), la sécurité des données n'était pas la première priorité.

Afin d'assurer la protection des données sensibles stockées sur nos divers objets connectés, il devient essentiel de trouver des moyens de chiffrement adaptés. L'algorithme ASCON s'avère alors être un outil de chiffrement de données adapté pour réaliser cette tâche.

Présentation de l'algorithme ASCON

ASCON est un algorithme de chiffrement authentifié avec des données associées. Il permet de réaliser un chiffrement dit "léger", c'est à dire optimisé pour les dispositifs à ressources limitées tels que les IoT pour garantir la confidentialité des données, afin de proposer une crypter des messages tout en s'assurant qu'ils n'ont pas été modifiés, grâce à un système d'authentification. En 2023, l'algorithme ASCON est élu par l'Institut National des Normes et de la Technologie (NIST) comme une référence de cryptographie légère, notamment pour les IoT.

Déroulé du projet de conception de l'algorithme ASCON

Pour réaliser un tel algorithme, il nous allons donc réaliser les étapes suivantes :

- Compréhension globale du fonctionnement de l'algorithme ASCON
- Compréhension théorique de chaque bloc élémentaire de l'algorithme, que nous détaillerons ultérieurement.
- Implémentation de l'algorithme en SystemVerilog, qui nous permettra de créer un code réalisant un chiffrement authentifié des données.
- Tests de vérification, afin de vérifier si les résultats obtenus sont cohérents.

Ainsi, nous allons donc pouvoir expliquer le processus de chiffrement ASCON-AEAD128.

Fonctionnement global de l'algorithme de chiffrement ASCON

Dans notre version de l'algorithme, il n'est question d'opérer que sur un état courant, nommé *State* S , qui est mis à jour au fur et à mesure de l'exécution de l'algorithme. L'état S est alors composé de 320 bits que l'on peut diviser en 5 registres S_i de 64 bits chacun.

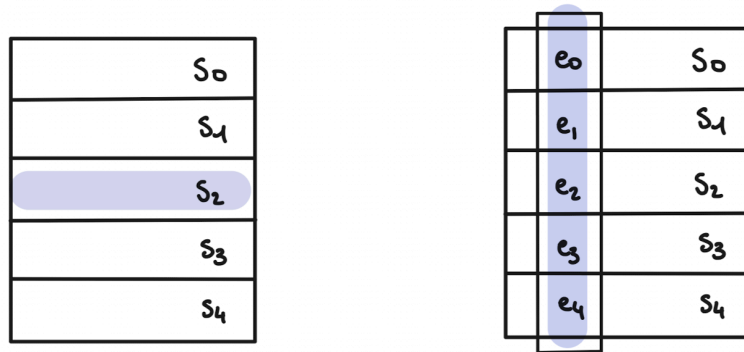


Figure 1: Représentation de l'état courant S en 5 registres de 64 bits (à gauche) et en 64 colonnes de 5 bits (à droite)

Le processus de chiffrement ASCON se décompose ensuite en 4 étapes clés, représentées sur la figure ci-dessous.

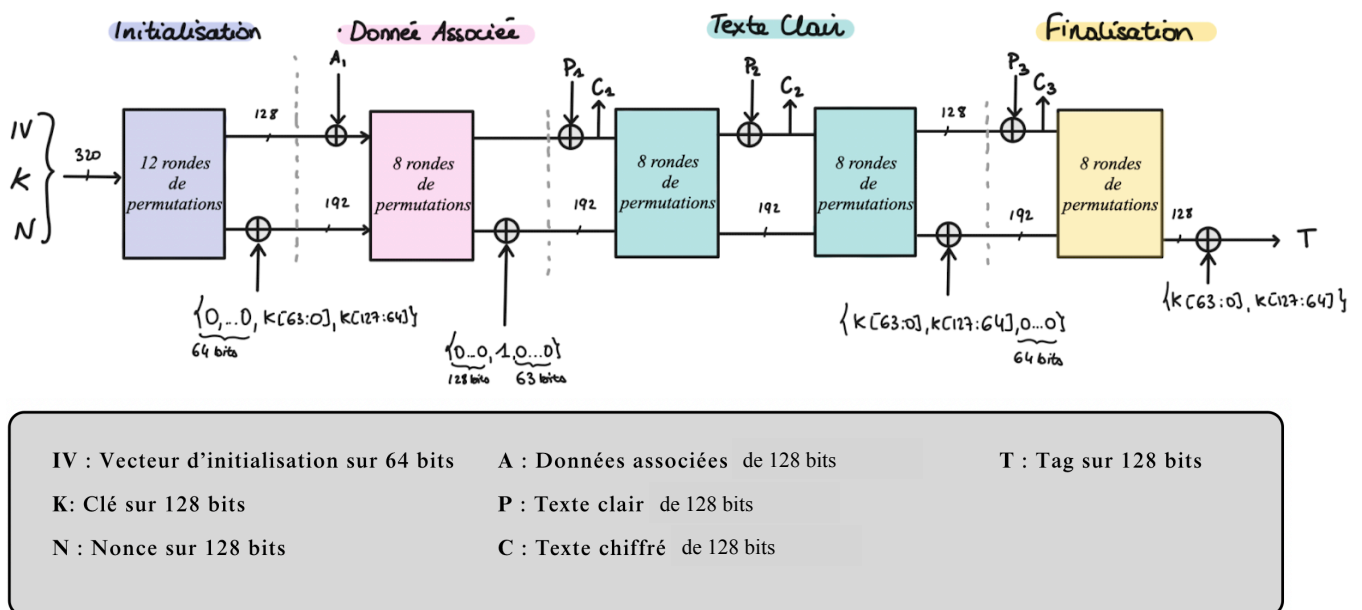


Figure 2 : Schéma du chiffrement ASCON-AEAD128

L'**initialisation** est la première phase de l'algorithme. Elle permet, comme son nom l'indique, d'initialiser l'état S en fonction du vecteur d'initialisation IV donné, de la clé K et d'un nombre arbitraire N , qu'on appelle *nonce*.

La seconde étape permet d'introduire la **donnée associée A**, et consiste essentiellement à additionner un bloc de données de 128 bits. Cette étape permet de s'assurer que le message chiffré ne soit pas remplacé par un autre de même clé.

Le traitement en **texte clair** est l'étape qui réalise l'essentiel du chiffage du message. On réalise ce traitement par blocs de 64 bits en chiffrant le message étape par étape.

La dernière étape de l'algorithme, la **finalisation**, permet de générer puis de récupérer le tag T, qu'on utilise par la suite pour l'authentification du message.

L'architecture de l'algorithme ASCON comprend 4 blocs :

1. La machine d'état finis (FSM) qui génère les signaux des contrôles.
2. La permutation XOR qui comprend les 3 transformations élémentaires déjà évoquées.
3. Un compteur 4 bits pour compter le bon nombre de rondes de permutations à exécuter.
4. Un autre compteur pour le nombre de blocs de données du texte chiffré. Cependant, ce module ne sera pas pris en compte dans ce rapport, faute d'avoir réussi à l'implémenter.

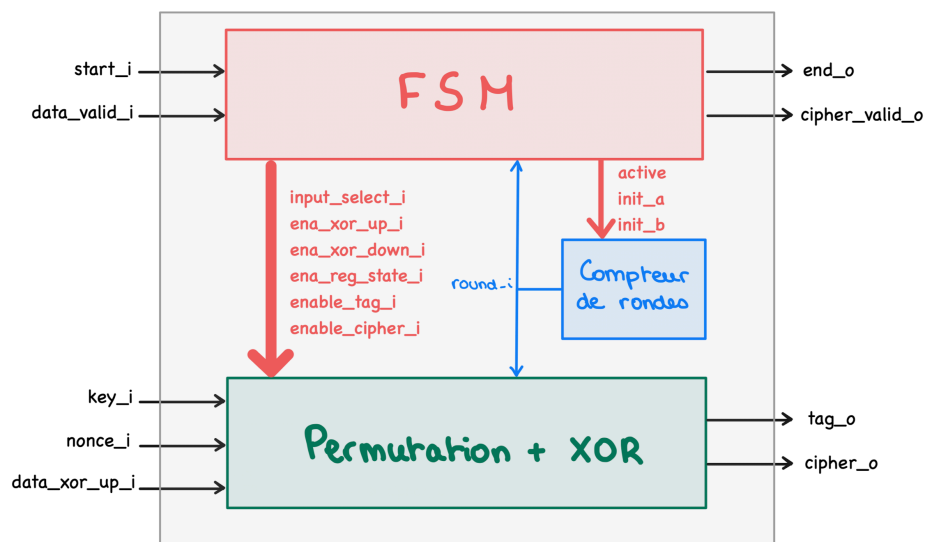


Figure 3 : Architecture Globale de l'algorithme ASCON

Les différentes entrées et sorties sont telles que :

Entrées	Sorties
L'horloge clk_i	Le signal end_o qui indique la fin du chiffrement
Le signal d'initialisation resetb_i	cipher_valid_o qui indique le changement de la sortie cipher_o
Le signal start_i qui permet de commencer le chiffrement	Le tag tag_o sur 128 bits
L'entrée data_valid_i qui indique le changement des données de data_xor_up_i	Le signal cipher_o sur 128 bits
La clé key_i sur 128 bits	
Le nombre arbitraire nonce_i de 128 bits	
L'entrée data_xor_up_i de 128 bits	

Modélisation de l'algorithme ASCON

Modélisation des fonctions élémentaires de la permutation

Chaque permutation est réalisée à l'aide de 3 opérations qui sont l'ajout de la constante sur un des vecteurs de l'état S , la couche de substitution appliquée en parallèle sur chaque colonne de 5 bits de S et la couche de diffusion.

Ainsi, la permutation est telle que $p = p_L \circ p_S \circ p_C$ avec p_C l'addition de la constante, p_S la substitution et p_L la couche de diffusion.

Addition de la constante p_C

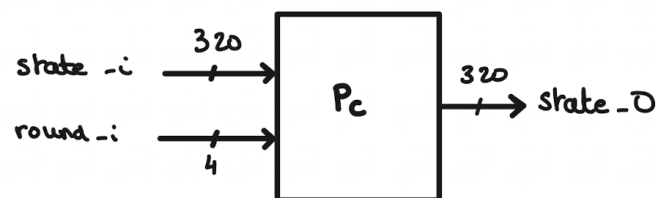


Figure 4 : Module d'addition de la constante P_C

L'ajout de la constante constitue la première étape de la permutation. Cette opération consiste à ajouter une constante de ronde, notée cr au registre S_2 de l'état interne. La valeur de cette constante dépend de la valeur du signal **round_i**, c'est-à-dire qu'elle dépend du nombre de tours déjà effectués dans la permutation.

Ainsi l'ajout de la constante se fait comme montré sur la figure ci-contre :

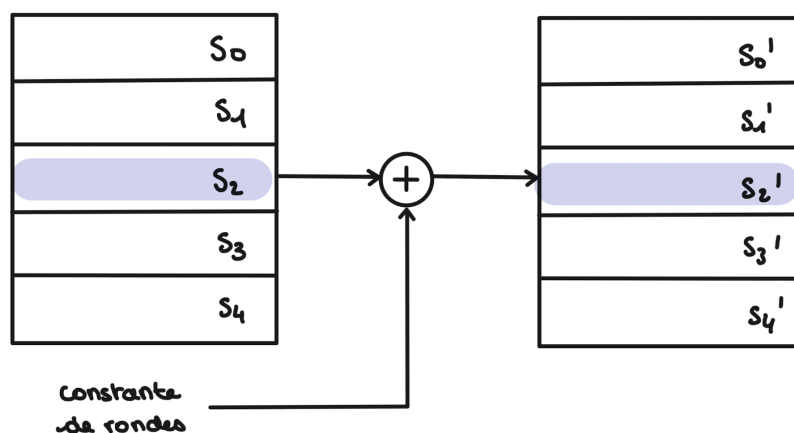


Figure 5 : Principe du module d'addition de la constante

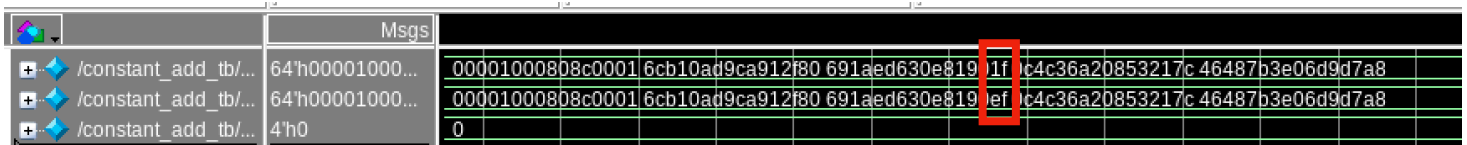
La nouvelle valeur du registre S_2 est donc déterminée à l'aide d'une opération de type XOR.

Le nombre de rondes et les constantes associées sont définies dans la table ci-dessous.

Ronde r de p^{12}	Ronde r de p^8	Constante c_r
0		0000000000000000f0
1		0000000000000000e1
2		0000000000000000d2
3		0000000000000000c3
4	4	0000000000000000b4
5	5	0000000000000000a5
6	6	000000000000000096
7	7	000000000000000087
8	8	000000000000000078
9	9	000000000000000069
10	10	00000000000000005a
11	11	00000000000000004b

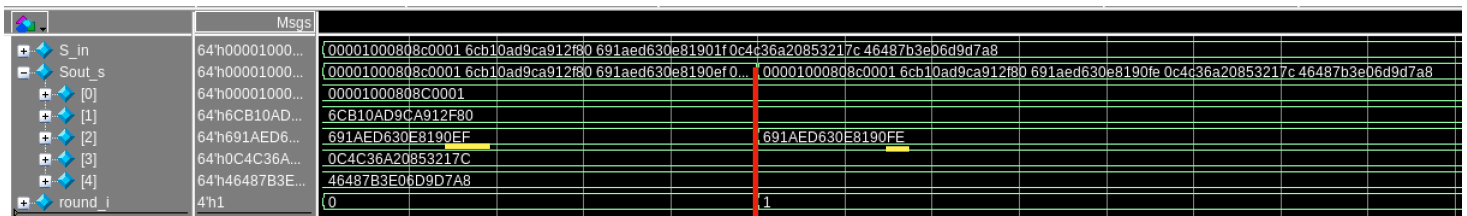
Table 1 : Addition de la constante pour p^8 et p^{12}

Ces valeurs sont également présentes dans le fichier **ascon_pack.sv**, dans le tableau **round_constant** qui nous permet d'avoir la valeur de la constante adaptée à la permutation réalisée.



Msgs	Data
/constant_add_tb/...	64h00001000...
/constant_add_tb/...	64h00001000...
/constant_add_tb/...	4h0

Figure 6 : Simulation de l'addition de la constante



Msgs	Data
S_in	64h00001000...
Sout_s	64h00001000...
[0]	64h00001000...
[1]	64h6CB10AD...
[2]	64h691AED6...
[3]	64h0C4C36A...
[4]	64h46487B3E...
round_i	4h1

Figure 7 : Simulation de l'addition de la constante pour deux rondes

La simulation nous montre qu'à chaque ronde, la valeur du second registre est modifiée en fonction de celle de **round_i**. Les autres lignes de l'état courant reste inchangées, on voit qu'elles ne sont pas modifiées lorsque que **round_i** est incrémenté.

En comparant les résultats de cette simulation avec ceux attendus, on peut ainsi en conclure que notre module d'addition de la constante semble correct.

Couche de Substitution ps

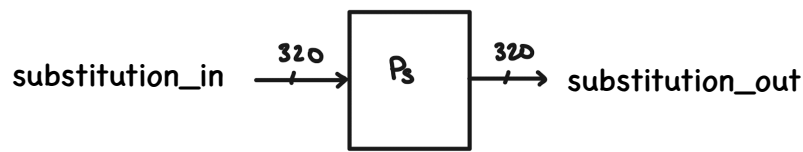


Figure 8 : Module de la couche de substitution P_s

La couche de substitution permet de mettre à jour l'état courant S , qu'on interprète comme un tableau de 64 colonnes de 5 bits. L'opération permet alors d'appliquer la substitution à chaque colonne de l'état courant

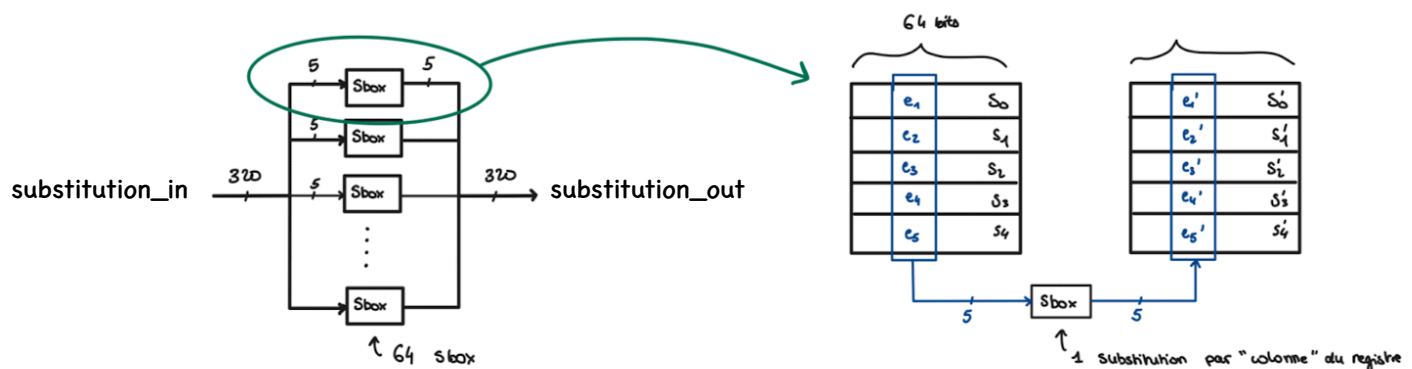


Figure 9 : Principe du module de substitution

La table de substitution utilisée est représentée dans la table 2. Pour ce projet, un module intermédiaire **sbox.sv** a été implémenté afin d'accéder aux valeurs nécessaires.

x	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
$S_{box}(x)$	04	0B	1F	14	1A	15	09	02	1B	05	08	12	1D	03	06	1C	1E	13	07	0E	00	0D	11	18	10	0C	01	19	16	0A	0F	17

Table 2 : Table de substitution

substitution_in	64'h00001000...	00001000808C0001 6CB10AD9CA912F80 691AED630E8190EF 0C4C36A20853217C 46487B3E06D9D7A8
substitution_out	64'h25f7c341c...	25f7c341c45f9912 23b794c540876856 B85451593D679610 4FAFBA264A9E49BA 62B54D5D460ADED4

Figure 10 : Simulation de la substitution

Avec **substitution_in**= 00001000808C0001 6CB10AD9CA912F80 691AED630E8190EF 0C4C36A20853217C 46487B3E06D9D7A8

On obtient **substitution_out** = 25F7C341C45F9912 23B794C540876856 B85451593D679610 4FAFBA264A9E49BA 62B54D5D460ADED4

Cela correspond bien au résultat attendu pour la couche de substitution.

Couche de diffusion linéaire pl

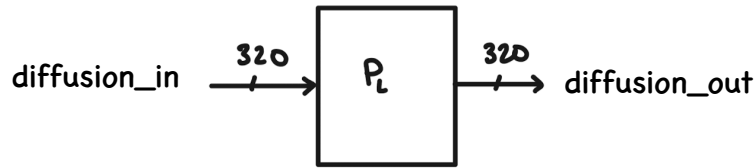


Figure 11 : Module de la couche de diffusion linéaire pl

La couche de diffusion linéaire est la dernière étape de la permutation. Elle permet de modifier l'état courant S en appliquant une diffusion à chacune de ses lignes.

$$\begin{aligned}
 S_0 &\leftarrow \Sigma_0(S_0) = S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28) \\
 S_1 &\leftarrow \Sigma_1(S_1) = S_1 \oplus (S_1 \ggg 61) \oplus (S_1 \ggg 39) \\
 S_2 &\leftarrow \Sigma_2(S_2) = S_2 \oplus (S_2 \ggg 1) \oplus (S_2 \ggg 6) \\
 S_3 &\leftarrow \Sigma_3(S_3) = S_3 \oplus (S_3 \ggg 10) \oplus (S_3 \ggg 17) \\
 S_4 &\leftarrow \Sigma_4(S_4) = S_4 \oplus (S_4 \ggg 7) \oplus (S_4 \ggg 41)
 \end{aligned}$$

Figure 12 : Opérations de la couche de diffusion linéaire

L'opération effectuée sur chaque ligne de l'état courant, $S_i \ggg X$, est en fait une rotation cyclique de X bits vers la droite. Cette rotation est également accompagnée par des XOR. Le module **diffusion_layer.sv** nous permet de réaliser correctement cette diffusion linéaire, en effectuant la rotation cyclique puis les XOR sur chaque registre de l'état courant.

diffusion_in	64'h25f7c341c...	25f7c341c45f9912	23b794c540876856	b85451593d679610	4fafba264a9e49ba	62b54d5d460aded4
[0]	64'h25F7C34...	25F7C341C45F9912				
[1]	64'h23B794C5...	23B794C540876856				
[2]	64'hB8545159...	B85451593D679610				
[3]	64'h4FAFBA2...	4FAFBA264A9E49BA				
[4]	64'h62B54D5...	62B54D5D460ADED4				
diffusion_out	64'h932c16dd...	932c16dd634b9585	b48a3c3fe8fb45ce	a69f28b0c721c340	05e1761f1e1fcb67	64d322a896b791cf
[0]	64'h932C16D...	932C16DD634B9585				
[1]	64'hB48A3C3...	B48A3C3FE8FB45CE				
[2]	64'hA69F28B...	A69F28B0C721C340				
[3]	64'h05E1761F...	05E1761F1E1FCB67				
[4]	64'h64D322A8...	64D322A896B791CF				

Figure 13 : Simulation de la diffusion linéaire

La simulation réalisée donne des résultats cohérents avec ceux attendus pour la couche de diffusion linéaire.

Modélisation de la permutation (sans XOR)

La permutation regroupe, comme déjà mentionné, les 3 opérations P_c , P_s et P_L . Avant de parler du fonctionnement de la permutation telle qu'utilisée dans l'algorithme ASCON, c'est à dire avec les opérations XOR, nous allons parler d'une modélisation simplifiée.

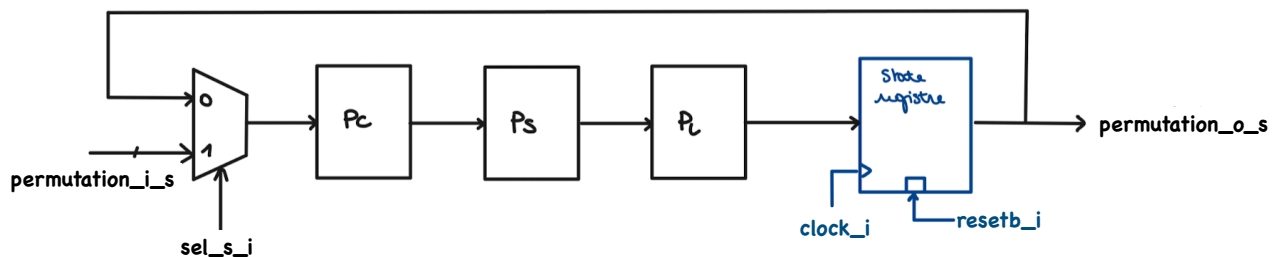


Figure 14 : Schéma de la permutation simplifiée (sans XOR)

Le circuit ci-dessus présente 5 instances différentes : le multiplexeur, les trois opérations élémentaires et un registre.

Le Multiplexeur

C'est la première instance du circuit, qui permet de déterminer quel signal va être transformé en fonction de la ronde de la permutation.

Le multiplexeur n'est "activé" qu'à la première ronde de permutation, avec **sel_s_i = 1**. Ensuite, vu qu'il faut exécuter 8 ou 12 rondes, il suffit de faire passer **sel_s_i** à 0 afin d'utiliser le résultat de la permutation précédente.

Le registre d'état state_registre

Dernière étape de cette permutation simplifiée, il permet de mémoriser la sortie des 3 opérations de la permutation afin de la réinjecter lors du cycle suivant.

Dans le module **state_registre.sv**, les différents signaux indiqués permettent de déterminer le signal de sortie du registre :

- si **reset_b_i = 0**, la sortie du registre est mise à zéro et donc **permutation_o** est nulle.
- si **reset_b_i = 1**, deux cas sont possibles :
 - **enable_p_i = 1** aussi et le registre garde alors en mémoire la valeur de sortie de P_L grâce au signal **q_i**
 - **enable_p_i = 0**, et le registre prend la valeur de **q_i**.

Le signal **q_i** permet de garder en mémoire la valeur du registre durant les états d'attente de la FSM.

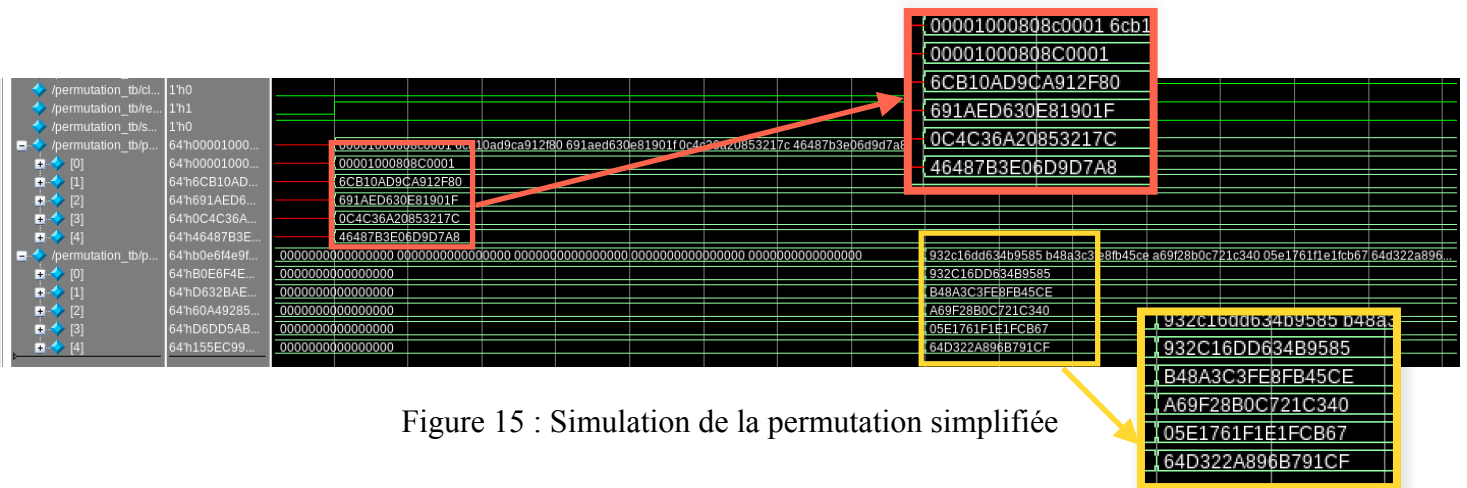


Figure 15 : Simulation de la permutation simplifiée

L'entrée **permutation_i** ainsi que la sortie **permutation_o** sont ici arbitraires. Dans l'algorithme ASCON complet, elles dépendent de la phase où on se situe (Initialisation, Donnée Associée, Texte Clair 1-2, Finalisation...) mais, pour les simulations, on a pris **permutation_i** égal à la Valeur Initiale IV.

Ainsi, avec **permutation_i** = 00001000808C0001 6CB10AD9CA912F80 691AED630E81901F 0C4C36A20853217C 46487B3E06D9D7A8

On obtient **permutation_o** = 932C16DD634B9585 B48A3C3FE8FB45CE A69F28B0C721C340 05E1761F1E1FCB67 64D322A896B791CF

Cela correspond bien au signal attendu à la fin de la première permutation.

Modélisation de la permutation avec les XOR

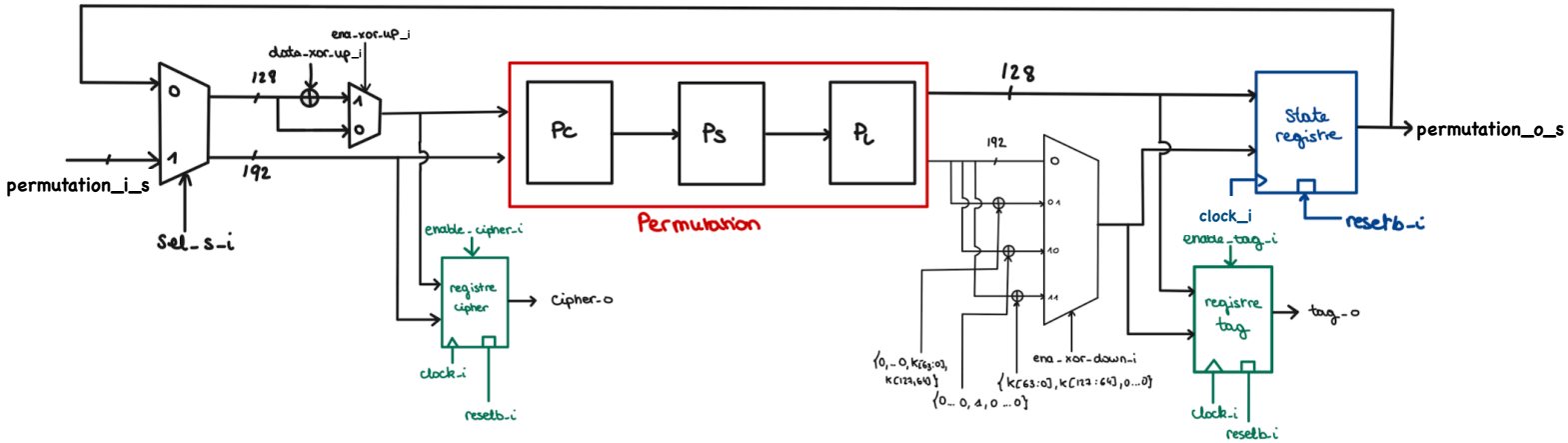


Figure 16 : Schéma de la permutation XOR

Cette représentation de la permutation correspond à celle plus classique décrite dans l'algorithme de chiffrement ASCON. On y retrouve les deux opérateurs XOR, en début et fin de permutation, ainsi que deux registres.

Ce circuit, plus complet et complexe que le précédent (cf. Figure 15), comprend 9 instances différentes : le multiplexeur d'entrée, le XOR UP, le registre cipher de 128 bits, les 3 opérations constituant le bloc de permutation, le XOR DOWN, un nouveau registre de 128 bits pour le tag et enfin le registre d'état.

Module XOR UP

Le premier XOR, qu'on a nommé **xor_up**, est utilisé avant les rondes de permutations. On l'utilise 4 fois dans notre algorithme de chiffrement : une fois au moment d'entrer dans les Données Associées, deux fois dans la phase de Texte Clair et une dernière fois en entrant dans la Finalisation.

Le module **xor_up.sv** comprend l'opérateur XOR ainsi que le multiplexeur qui suit.

Lorsqu'un XOR doit être effectué, **ena_xor_up_i** vaut 1. Si **ena_xor_up_i** = 0, alors le signal qui arrive sur le registre cipher et la permutation n'est pas "xoré".

Le signal **data_xor_up_i** correspond successivement aux valeurs A1, P1, P2 et P3 en fonction de la phase de chiffrement en cours.

Registre Cipher

Placé en sortie du XOR UP, le registre cipher permet de conserver les valeurs C1, C2 et C3 du Texte Chiffré.

On utilise ce registre trois fois seulement : deux fois dans la phase de Texte Clair (en entrée et entre les deux rondes de permutations) et une fois en entrée de la phase de Finalisation.

Lorsque l'entrée **enable_cipher_i** vaut 1, le registre est mis à jour, ce qui permet la conservation du Texte Chiffré sur le signal **cipher_o**.

Module XOR DOWN

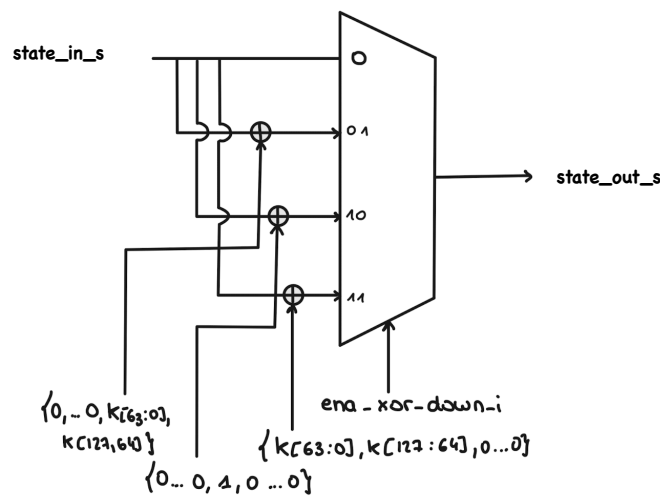


Figure 17 : Zoom sur le fonctionnement du XOR DOWN

Situé après la permutation, le module **xor_down** comprend l'opération XOR et un multiplexeur. En fonction des valeurs de **ena_xor_down_i**, le signal envoyé dans les registres du tag et du cipher seront différents.

En revenant sur le schéma du chiffrement ASCON (cf. Figure 3), on voit que le XOR apparaît 4 fois dans l'algorithme : une fois à la fin de l'initialisation, une autre à la sortie des Données Associées, une seule fois en sortie du Texte Clair et à la toute fin de la simulation (en fin de la Finalisation). Lors de ces 4 opérations, le signal envoyé sur l'opérateur XOR ne change que 3 fois car il est identique pour les première et dernière fois. Ainsi, lors de l'implémentation du module, il a suffi de créer ces 4 cas (puisqu'on inclut celui où il n'y a pas de XOR à faire).

Le fonctionnement du module est tel que :

- **ena_xor_down_i** = 01 à la fin de l'Initialisation ;
- **ena_xor_down_i** = 10 en sortie des Données Associées ;
- **ena_xor_down_i** = 11 en sortie du Texte Clair ;
- **ena_xor_down_i** = 01 à la fin de la Finalisation
- **ena_xor_down_i** = 00 tout le reste du temps, c'est à dire au sein des rondes de permutations.

L'entrée **key_i**, figurant dans le code de **xor_down sv**, correspond au signal qui est envoyé sur l'opérateur, modifié en fonction de la phase.

Registre Tag

Placé en sortie du XOR DOWN, le registre cipher permet de conserver la valeur du Tag. Il n'est utilisé qu'à la toute fin de la simulation, à la dernière ronde de permutation de la Finalisation.

Lorsque l'entrée **enable_tag_i** vaut 1, le registre est mis à jour, ce qui permet la conservation du Tag sur le signal de sortie **tag_o**.

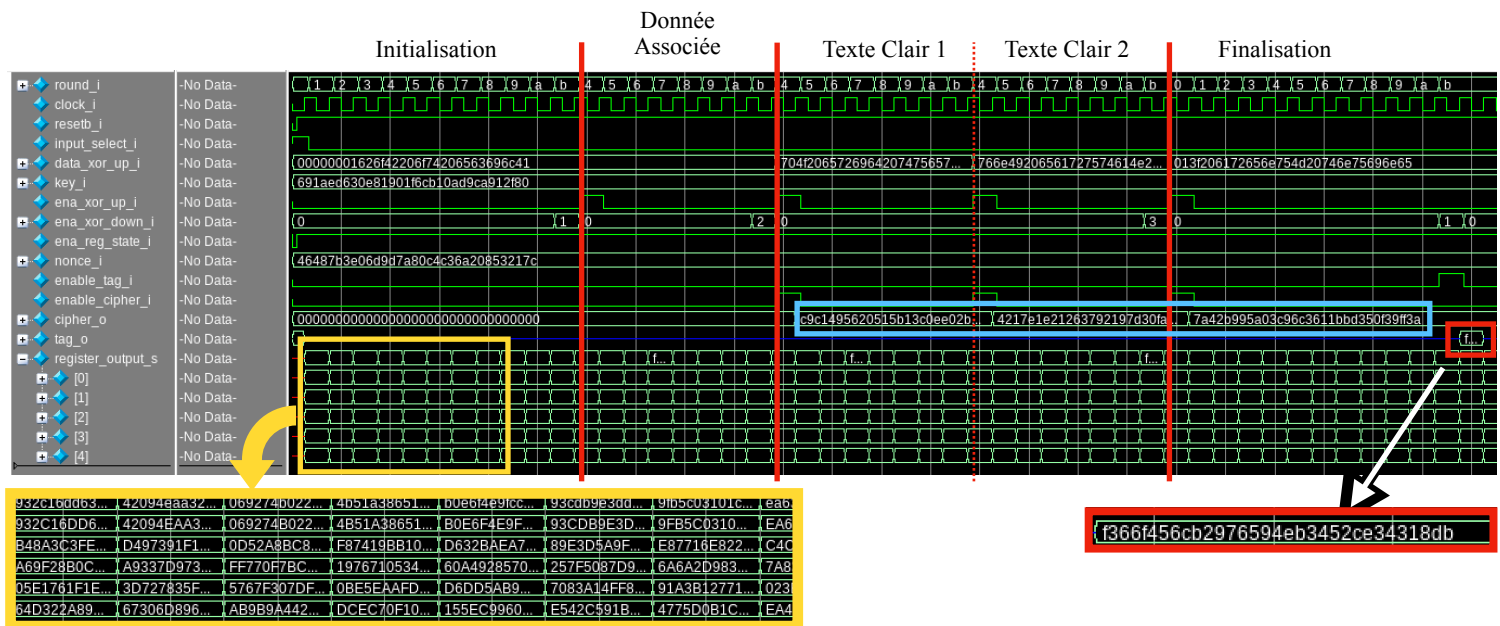


Figure 18 : Simulation de la permutation XOR

La simulation affiche ici le signal interne **register_output_s** qui détaille chaque étape de la permutation. L'encadré jaune montre ces signaux de sortie intermédiaires. On peut voir qu'ils correspondent bien à ceux attendus, validant alors le fonctionnement de notre permutation.

Cette simulation nous permet également de visualiser de Tag de sortie, encadré en rouge. En le comparant avec celui donné dans le sujet, on voit que les données correspondent.

Enfin, les signaux C1, C2 et C3 extraits du Texte Chiffré sont encadrés en bleu sur la figure ci-dessus, et sont bien cohérents avec ceux fournis.

On voit donc bien que le chiffrement avec la permutation XOR renvoie les données recherchées.

Modélisation de la machine d'état finis (FSM) - Machine de Moore

La machine d'états est le centre de l'algorithme ASCON. C'est elle qui permet de gérer les flux de données et les différentes phases de l'algorithme, en contrôlant ses signaux.

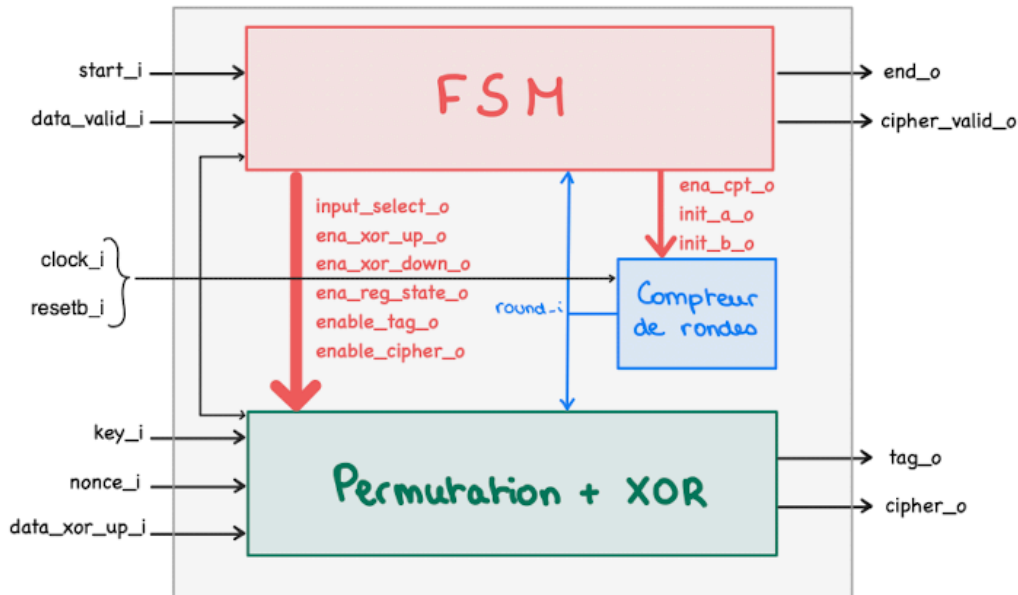


Figure 19 : Structure de l'algorithme ASCON

Les entrées de la machine d'états sont :

- **clock_i** : l'horloge.
- **resetb_i** : Permet de réinitialiser la FSM si nul.
- **start_i** : Signal de démarrage de la machine. Si **start_i** = 1, le chiffrement commence.
- **data_valid_i** : Indique lorsque le signal **data_xor_up_i** change au cours de la simulation. S'il n'est pas activé au bon moment, l'algorithme ne prend pas en compte les valeurs A, P1, P2, P3 ajoutées au XOR UP lors du chiffrement.
- **round_i** : Le compteur de rondes, qui est en fait une entrée interne de la FSM.

Les sorties de la machine d'états sont :

- **end_o** : Indique lorsque l'algorithme est terminé.
- **cipher_valid_o** : Indique si la valeur du cipher a bien été mise à jour. Cela nous permet de savoir où nous récupérerons les Textes Chiffrés C1, C2 et C3.
- **input_select_o** : Commande le multiplexeur d'entrée de la permutation avec le signal **sel_s_i**
- **ena_xor_up_o** : Signal d'activation du XOR UP
- **ena_xor_down_o** : Signal d'activation du XOR DOWN
- **ena_reg_state_o** : Active l'écriture dans le registre d'état **state_registre**
- **enable_tag_o** : Active l'écriture dans le registre du tag **registre_tag**
- **enable_cipher_o** : Active l'écriture dans le registre du cipher **registre_cipher**
- **ena_cpt_o** : Permet d'incrémenter le compteur **round_i**
- **init_a_o** : Initialise **round_i** à 0
- **init_b_o** : Initialise **round_i** à 4

Remarque : On peut voir ici que le compteur de blocs n'a pas été implémenté. Ayant préféré m'assurer du bon fonctionnement de la permutation et de la FSM, j'ai décidé de ne pas m'attarder sur cette partie de l'algorithme, qui n'est pas essentielle au processus de chiffrement.

Le fonctionnement de la machine d'état de Moore se décompose en plusieurs phases, relatives à celles du chiffrement ASCON, qui sont représentées sur le diagramme d'état ci-dessous.

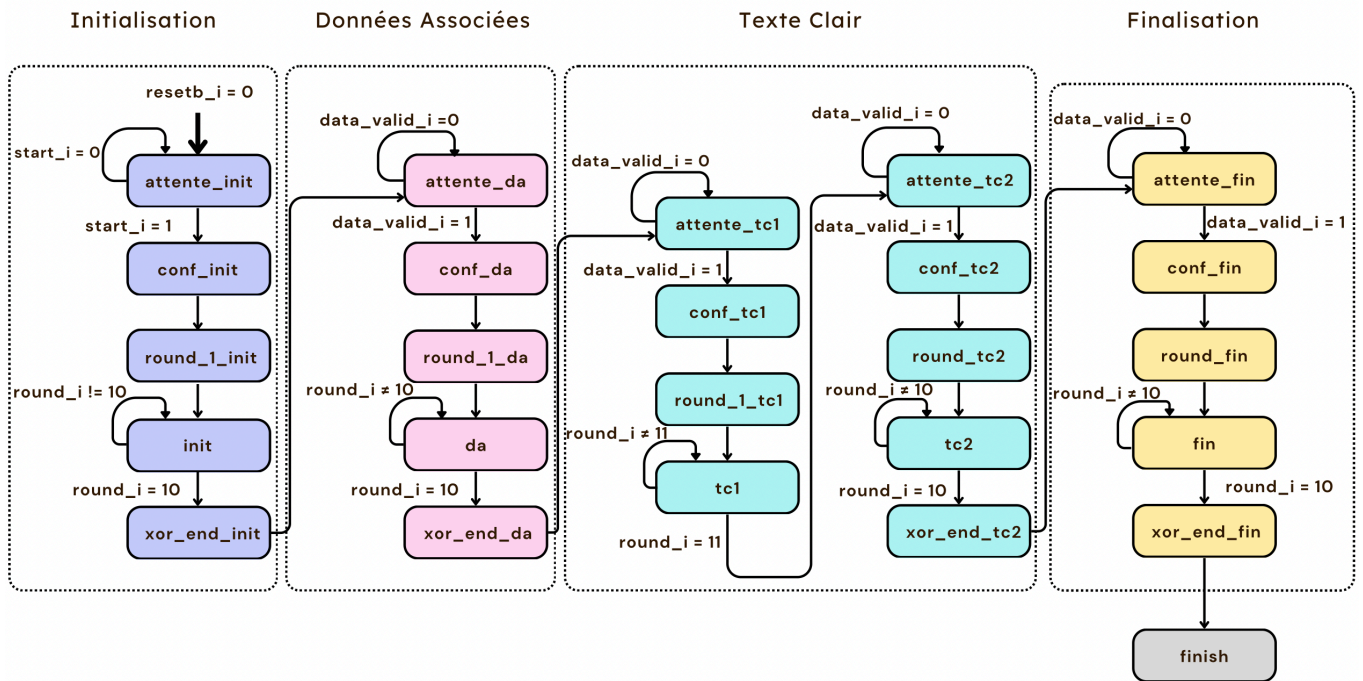


Figure 20 : Diagramme d'état de la machine d'états finis

Les couleurs de chaque blocs sont relatives à celles figurant sur le schéma du chiffrement de la Figure 3 (cf. page 4). Dans chaque bloc, on retrouve les états suivant :

- attente_XXX : L'état d'attente dans lequel la machine d'état reste s'il ne reçoit pas le signal de validation attendu (**start_i** ou **data_valid_i**). A cet état, on garde la valeur **q_i**.
- conf_XXX : État d'initialisation du compteur **round_i** pour réaliser les 8 ou 12 rondes de permutation. À l'initialisation et la finalisation, **round_i = 0**, sinon **round_i = 4**.
- round_1_XXX : Ce bloc indique qu'un XOR UP est effectué. A noter qu'aucun XOR UP n'est utilisé à l'initialisation mais on utilise ce bloc pour permettre au multiplexeur de la permutation de choisir le bon signal d'entrée.
- init, da, tc1, tc2, fin : États où on réalise les rondes de permutation nécessaires au chiffrement
- xor_end_XXX : Indique qu'un XOR DOWN est réalisé après la dernière permutation de la ronde.

Une fois la machine d'états implémentée, nous pouvons dresser a table de vérité suivante, qui indique les valeurs des entrées/sorties du module **fsm**.

		Compteur de Rondes			Permutation						Sorties	
		ena_cpt_o	init_a_o	init_b_o	input_select_o	ena_reg_state_o	ena_xor_up_o	ena_xor_down_o	ena_cipher_o	ena_tag_o	end_o	cipher_valid_o
Initialisation	attente_init	0	0	0	0	0	0	0	0	0	0	0
	conf_init	0	1	0	0	0	0	0	0	0	0	0
	round_1_init	1	0	0	1	1	0	0	0	0	0	0
	init	1	0	0	0	1	0	0	0	0	0	0
	xor_end_init	1	0	0	0	1	0	01	0	0	0	0
Données Associées	attente_da	0	0	0	0	0	0	0	0	0	0	0
	conf_da	1	0	1	0	0	0	0	0	0	0	0
	round_1_da	1	0	0	0	1	1	0	0	0	0	0
	da	1	0	0	0	1	0	0	0	0	0	0
	xor_end_da	1	0	0	0	1	0	10	0	0	0	0
Texte Clair	attente_tc1	0	0	0	0	0	0	0	0	0	0	0
	conf_tc1	1	0	1	0	0	0	0	0	0	0	0
	round_1_tc1	1	0	0	0	1	1	0	1	0	0	1
	tc1	1	0	0	0	1	0	0	0	0	0	0
	attente_tc2	0	0	0	0	0	0	0	0	0	0	0
	conf_tc2	1	0	1	0	0	0	0	0	0	0	0
	round_1_tc2	1	0	0	0	1	1	0	1	0	0	1
	tc2	1	0	0	0	1	0	0	0	0	0	0
xor_end_tc2	1	0	0	0	1	0	11	0	0	0	0	
Finalisation	attente_fin	0	0	0	0	0	0	0	0	0	0	0
	conf_fin	1	0	1	0	0	0	0	0	0	0	0
	round_1_fin	1	0	0	0	1	1	0	1	0	0	1
	fin	1	0	0	0	1	0	0	0	0	0	0
	xor_end_fin	1	0	0	0	1	0	01	0	1	1	0
	finish	0	0	0	0	0	0	0	0	0	0	0

Table 3 : Table de vérité de la machine d'état

Le dernier module à implémenter avant de terminer la modélisation est l'ascon top. Il permet de s'assurer que chaque entrées et sorties sont liées aux bons modules associés. Grossièrement, c'est la "boîte" dans laquelle on range et connecte chaque bloc de l'algorithme. Sur le schéma de la figure 19 (cf. page 16), on l'assimilerait au carré gris dans lequel tous les autres blocs sont placés.

Résultats d'implémentation

La modélisation terminée, il ne restait plus qu'à lancer la simulations pour vérifier la cohérence des résultats avec ceux désirés

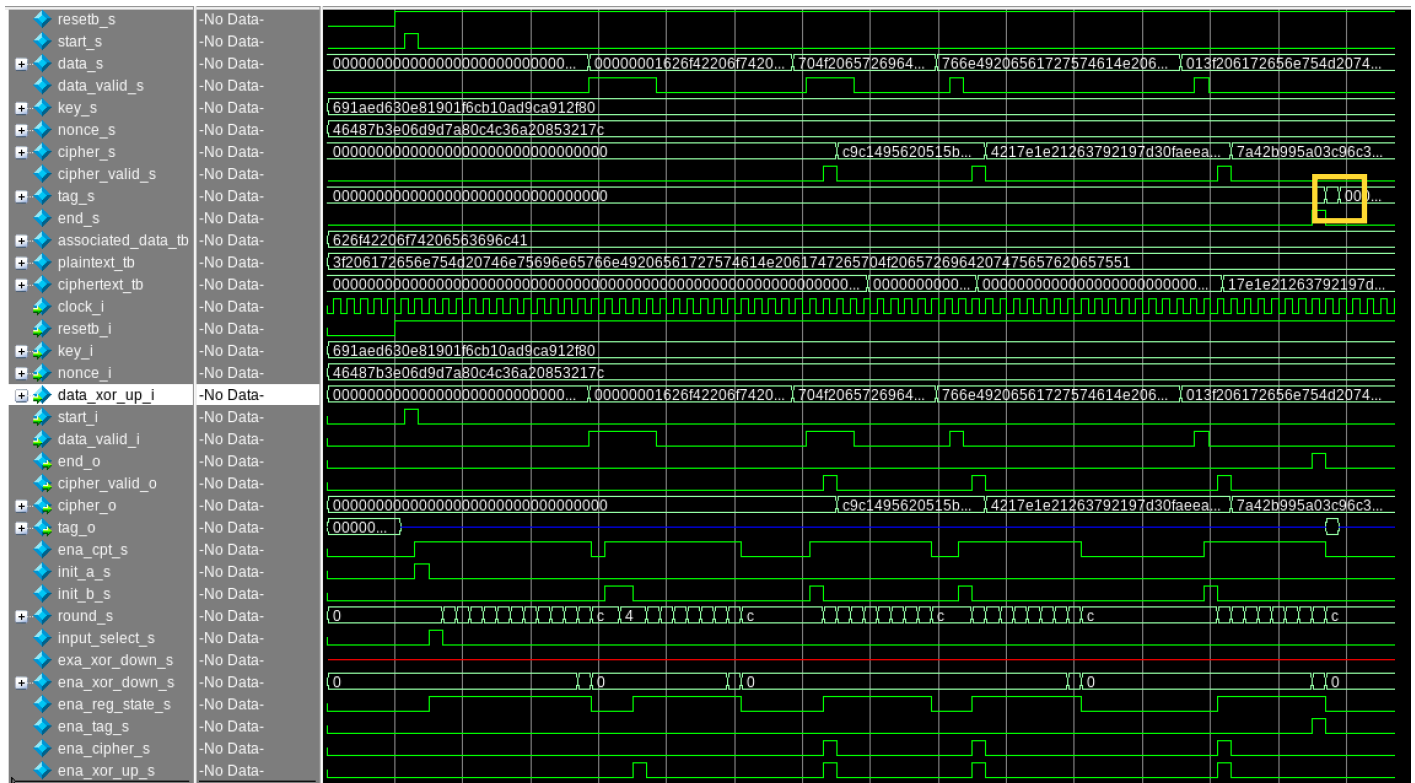


Figure 21 : Simulation de la machine d'état

La simulation de la FSM renvoie les résultats attendus pour au niveau des résultats de la permutation et des signaux extraits du Texte Clair C1, C2 et C3. Ces signaux étant identiques à ceux mis en avant lors de la simulation de la permutation XOR, on ne les remettra pas une nouvelle fois.

Le seul problème de cette machine d'état est que le Tag , qu'on trouve au niveau de l'encadré jaune sur la figure ci-dessus, récupéré en fin de simulation ne correspond pas à celui recherché.

Cependant, la simulation du module **permutation_xor** nous renvoyait le bon tag, ce qui s'avère étonnant.

```

1 -> 470bee3fe021df785b8cc07303b4e35a
2 -> f366f456cb2976594eb3452ce34318db

```

Figure 22 : En haut (1), le tag obtenu à l'issue de la FSM
En bas (2), le tag en sortie du module **permutation_xor**

Malheureusement, je n'ai pas réussi à trouver où était l'erreur dans la simulation.

Difficultés rencontrées

Lors de la réalisation de ce projet, il m'a fallu faire face à plusieurs "obstacles" pour arriver à une modélisation fonctionnelle.

Erreurs de Syntaxe

Lors de l'implémentation de l'algorithme ASCON, j'ai de nombreuses fois fait face à des erreurs de syntaxe dans mon programme. Ces dernières étaient des fois vraiment mineures : il suffisait qu'une virgule soit mal placée ou qu'un point-virgule remplace deux-points pour fausser tout un module. De plus, il arrivait que le code **compile_ascon** ne compile pas le bon module pour mettre des erreurs partout. Heureusement, ces problèmes étaient résolus facilement une fois qu'on trouvait le module et la ligne qui faisaient défaut à la compilation du programme.

La permutation

La permutation est ce qui m'a posé le plus de problèmes dans ce projet. Si l'implémentation des fonctions élémentaires Pc, Ps et PL ne m'a pas posé de gros problèmes, la mise en commun de ces 3 fonctions et la compréhension du fonctionnement des XOR m'ont fait énormément défaut. Malheureusement, j'ai dû passer plus de temps que prévu sur ce module afin de m'assurer de bien le comprendre et de bien l'implémenter. J'ai recommencé plusieurs fois l'écriture des deux modules **permutation** et **permutation_xor** avant d'arriver à quelque chose de fonctionnel.

Si j'ai mis aussi longtemps pour terminer le bloc de permutation, c'est parce que je voulais m'assurer de vraiment comprendre son fonctionnement isolé et au sein de l'algorithme ASCON. En effet, lorsque je n'arrive pas à voir la logique d'une fonction, j'avance beaucoup moins bien, donc c'était pour moi essentiel de maîtriser le module.

La Machine d'états finis

Étonnamment, l'implémentation de la FSM a été assez rapide. Je pensais initialement sauter cette étape par manque de temps mais, ayant passé suffisamment de temps sur la permutation et son fonctionnement général, j'ai assimilé plutôt rapidement le fonctionnement de la FSM qui me semblait finalement assez logique.

Debug des modules

Si la compréhension finale de l'algorithme de chiffrement ASCON ne m'a pas posé énormément de problèmes, le processus de débogage des modules **ascon_top**, **fsm** et **permutation_xor** s'est avéré assez lent.

Comme déjà énoncé, la permutation XOR m'a pris le plus de temps à comprendre mais aussi à debug. C'est en prenant le temps de bien lire et comprendre les résultats de la simulation que j'ai pu identifier les erreurs dans mon programme.

En ce qui concerne la FSM et l'ASCON Top, c'est en comparant la simulation avec les résultats donnés dans le sujet que j'ai (approximativement) trouvé mes erreurs. Cependant, je n'ai pas réussi à récupérer le Tag attendu en fin de chiffrement. Ce dernier apparaît cependant bien à la fin de ma permutation XOR, ce qui m'a étonné.

Conclusion

Ce projet, bien que très intéressant, a été pour moi très fastidieux. Puisque j'ai d'abord eu du mal à comprendre le premier gros bloc de l'algorithme, j'ai été très vite découragée. Cependant, en prenant plus de recul sur le sujet et le fonctionnement de l'algorithme j'ai pu progresser très rapidement et comprendre la logique du système.

J'aurai aimé être capable de récupérer le bon Tag en fin de simulation mais, par manque de temps et de moyens, cela n'était pas possible. Je trouve cela dommage, d'autant plus qu'il était en fin de ma permutation XOR. Aussi, il aurait été intéressant de réussir à implémenter le compteur de blocs, afin d'optimiser le fonctionnement de l'algorithme mais le projet m'avait semblé trop dense pour aborder cette notion sans perdre plus de temps que je n'en avais déjà perdu sur la permutation.

Le sujet en lui même touche à une thématique qui m'intéresse tout particulièrement : la cryptographie. Ayant déjà réalisé des travaux sur ce domaine et son historique, j'ai trouvé particulièrement intéressant de me plonger dans le fonctionnement d'une machine de chiffrement. Aussi, le futur de l'IoT me permet de constater l'importance nouvelle accordée à la cryptographie et à ses applications.